VERILOG · TEAM PROJECT 04

UART + FIFO Loopback

설계 · 시뮬레이션 검증



FIFO DEPTH 4 · push → wptr, pop → rptr

clk

100 MHz

Basys-3

baud

9600 bps

8N1 · LSB first

buffer

FIFO 4 × 2

FIFO_RX · FIFO_TX

검증

S1 ~ S5 PASS

시나리오 5개

정안성

Contents

uart_rx와 uart_tx 사이 FIFO 2단
연속 입력 echo 검증

00 **Summary** — 핵심 수치 · 시나리오 결과 — p. 3

01 **목적** — FIFO 추가 목적 · 목표 — p. 4

02 **UART Reference** — UART 통신 개념 · 16x oversampling · FIFO 개념 · 속도 관계 — p. 5 – 8

03 **블록 다이어그램** — uart_fifo_loopback · fifo 내부 — p. 9 – 10

04 **동작 설계** — control_unit 동작표 · rx_done → tx 흐름 — p. 11 – 12

05 **시뮬레이션** — TB 구조 · 시나리오 · S1 ~ S5 — p. 13 – 19

06 **트러블 슈팅** — 검증 중 확인한 경계 동작 — p. 20

07 **결론 · 고찰** — 결론 · 고찰 · 개선 방향 — p. 21

A **APPENDIX** — Margin · 16x 선택 이유 · ASM · TB 로그 — p. 23 – 27

00

UART + FIFO Loopback 구현 · 검증 결과

결론 · uart_rx와 uart_tx 사이 FIFO_RX · FIFO_TX → 수신 byte가 FIFO 2단을 거쳐 순서대로 echo
· 시나리오 5개 모두 PASS — FIFO 단독 · 단일 byte · 9600 연속 · 최악 조건 · glitch 회귀

FIFO

DEPTH 4 × 2

FIFO_RX · FIFO_TX, 8 bit

baud (PC / FPGA)

9600 / 9600.6

PC frame ≥ TX frame 160 tick

S1 FIFO 단독

414 / 0

step / mismatch

S2 · S3 · S4 echo

256 / 256

단일 · 9600 연속 · 최악 조건

S5 START glitch

23 / 23

02 수정 결과 유지

FIFO 최대 점유 (RX / TX)

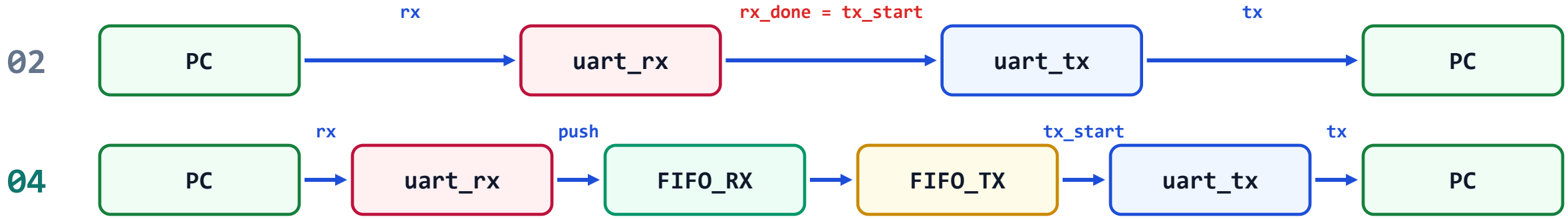
1 / 1

S2 ~ S4 · full 0회 · TX busy 중 rx_done 0회

TX가 바쁜 동안 들어온 Byte도 놓치지 않는 Loopback

01

결론 · 수신(uart_rx)과 송신(uart_tx) 사이에 FIFO_RX · FIFO_TX를 두어 두 쪽을 분리
· 연속 입력도 손실 · 중복 없이 같은 순서로 echo → 시나리오 5개로 검증



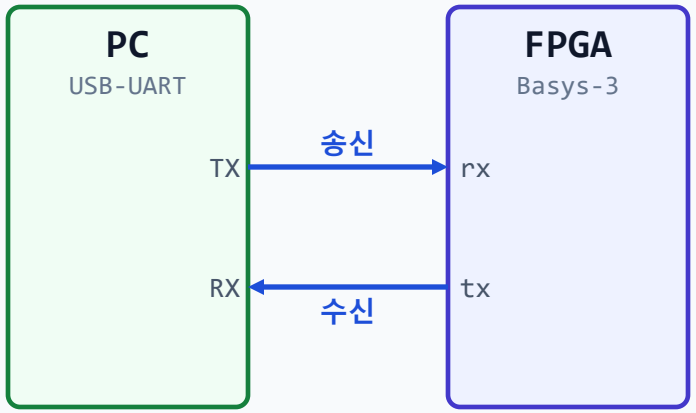
#	대상	목표 동작	결과
S1	**fifo**	push / pop / 동시 push·pop, full · empty, pointer 순환	**PASS**
S2	**uart_fifo_loopback**	단일 byte echo 유지	**PASS**
S3	**uart_fifo_loopback**	9600 bps 연속 입력 손실 · 중복 0	**PASS**
S4	**uart_fifo_loopback**	여유 0 tick 최악 조건에서도 손실 0	**PASS**
S5	**uart_rx**	START glitch 개선 유지	**PASS**

02

UART: Clock 없이 Start Bit로 시점을 맞추는 비동기 직렬 통신

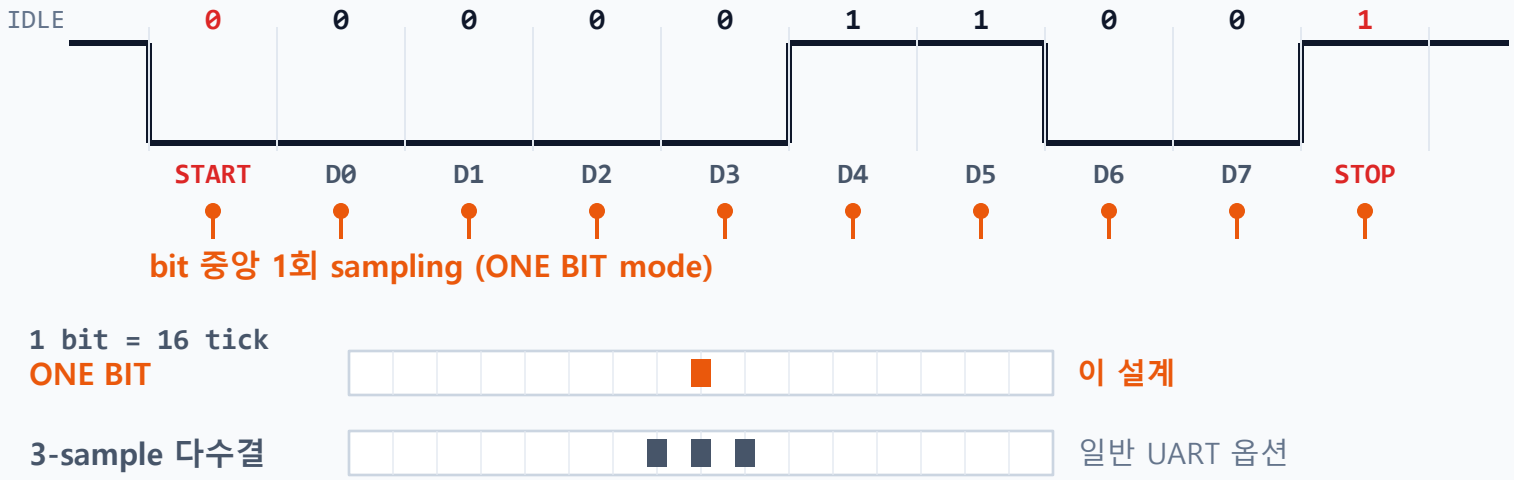
결론 · TX · RX 선이 따로 → 송신 · 수신 동시 가능 (full-duplex), 공유 clock 없음 → 양쪽이 baud만 약속
· 8N1 frame = START(0) → D0 ~ D7 (LSB first) → STOP(1), 이 설계는 bit 중앙 1회 sampling (ONE BIT mode)

연결



선 2개 → 동시 송수신

8N1 frame · '0' = 0x30



1 **Start bit** 1 → 0 falling edge = frame 시작
이 edge로 bit 경계 재정렬

2 **Stop bit** 항상 1 → 다음 falling edge 보장
오차가 frame마다 초기화

3 **LSB first** D0부터 송신 → 0x30은 0,0,0,0,1,1,0,0
RX는 MSB에 넣고 오른쪽 shift

4 **ONE BIT mode** bit 중앙 sample 1개로 판정
bit 중앙 glitch에 취약 (부록)

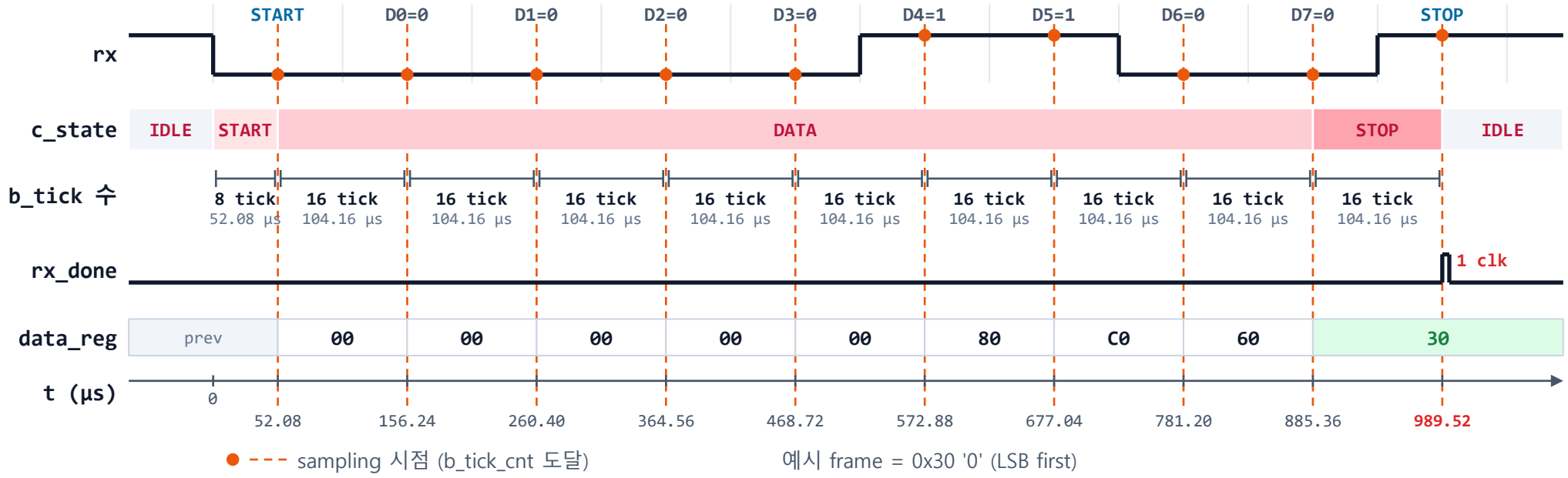
02

100 MHz ÷ 651 = b_tick, 16 tick = 1 bit (104.16 µs)

결론

- F_COUNT = 100 MHz ÷ (9600 × 16) = 651 → b_tick 6.51 µs, 1 bit = 16 tick = 104.16 µs (9600.6 bps)
- START 8 tick 대기로 bit 중앙 정렬 → 이후 16 tick마다 sampling, rx_done은 STOP 중앙

F_COUNT	b_tick 주기	1 bit	start 중앙
100 MHz ÷ 153,600 = 651	651 × 10 ns = 6.51 µs	16 tick = 104.16 µs	START 8 tick = 52.08 µs



xsim 측정 (tb_measure.sv)

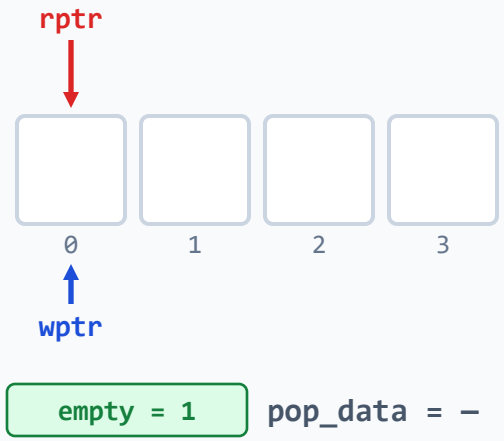
b_tick 주기	**6.51 µs**	START 체류	**52.08 µs**	sample 간격	**104.16 µs**
첫 sample	**rx fall + 156.245 µs**	rx_done	**rx fall + 989.525 µs**	TX 시작	**rx_done + 10 ns**

02

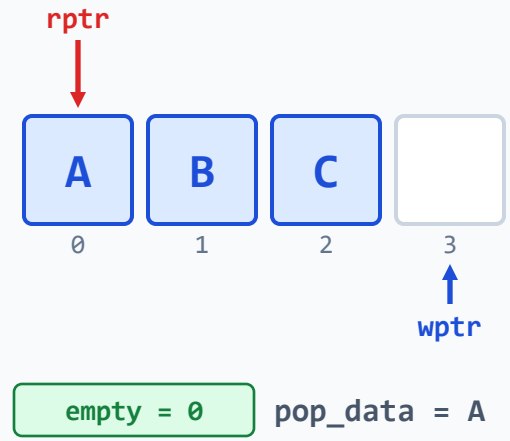
FIFO: wptr · rptr 두 포인터로 순서를 지키는 원형 버퍼

결론
- **push = wptr 자리에 쓰고 wptr + 1, pop = rptr + 1 → 두 포인터 비교로 full · empty 판단**
- **pop_data = mem[rptr] → pop 전에도 맨 앞 byte가 출력에 보임 (FWFT)**

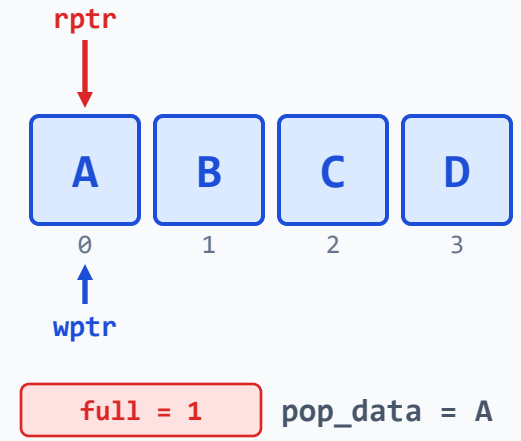
1 reset



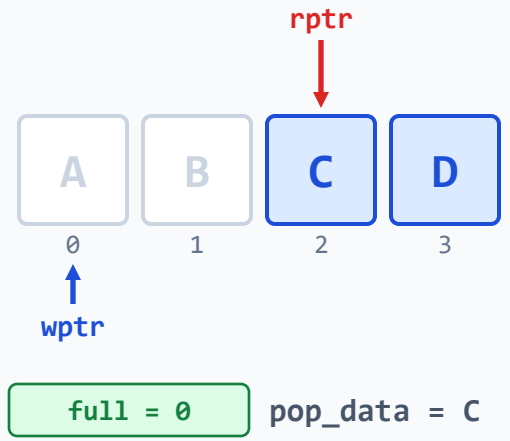
2 push A · B · C



3 push D



4 pop × 2



1 **full** push 후 wptr이 rptr을 따라잡음 → full ← 1

2 **empty** pop 후 rptr이 wptr을 따라잡음 → empty ← 1

3 **wptr = rptr** empty · full 둘 다 같음 → flag로 구분

4 **FWFT** 읽기 = 조합 출력 → TX가 바로 맨 앞 byte 사용

02

PC 9600 ≤ FPGA 9600.6 bps → FIFO는 대기 없이 통과

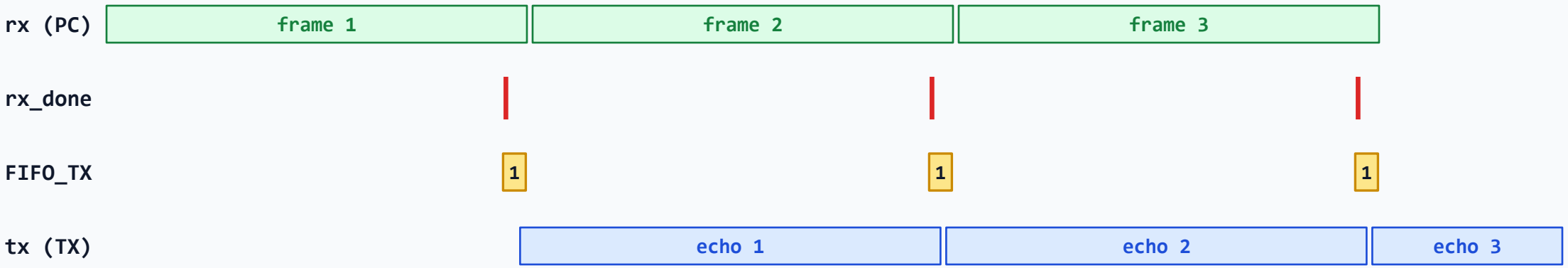
결론

- PC frame 1,041,666.7 ns ≥ TX frame 160 tick = 1,041,600 ns → FIFO가 계속 쌓이지 않음
- rx_done 순간 TX는 이미 IDLE (S3 · S4 TX busy 중 rx_done 0회) → FIFO는 대기 없이 통과만 (최대 1개)

10 bit frame 길이



연속 입력 개념도 (비율 무시)



rx_done 순간 앞 echo는 이미 종료 → byte는 FIFO 2단을 4 clk 동안 통과 → 다음 echo 바로 시작 (S4 frame gap 0 clk)

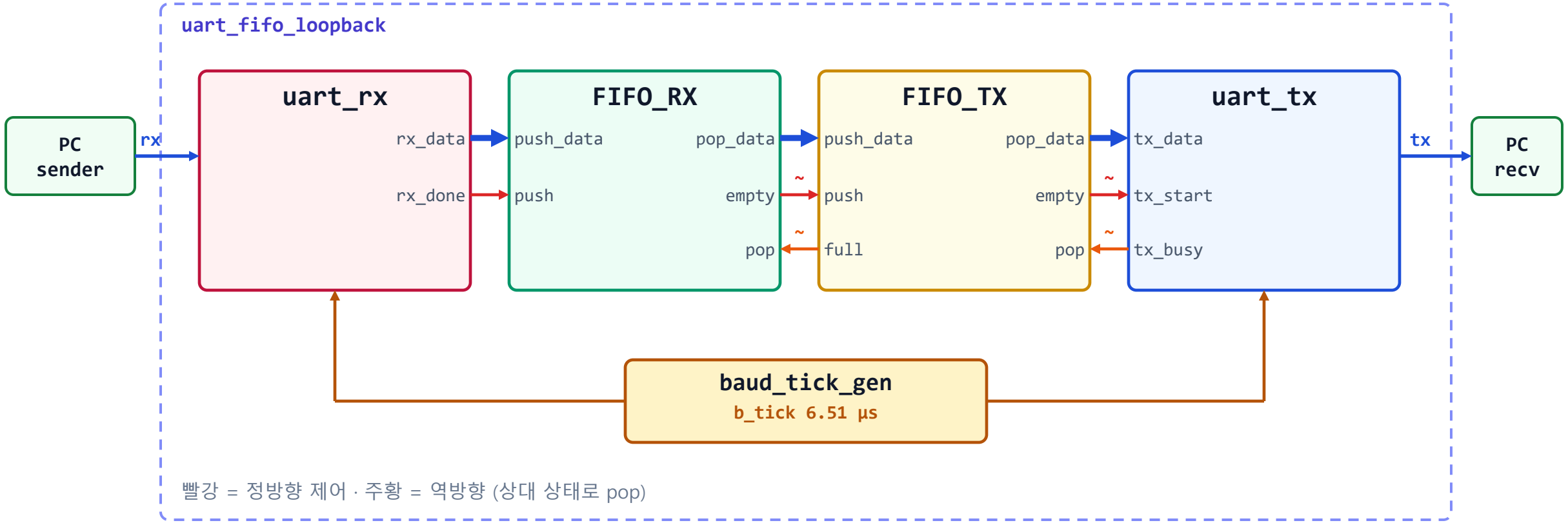
FIFO_RX · FIFO_TX 최대 점유 1 = 통과 중인 1 byte · full 0회

03

수신 → FIFO_RX → FIFO_TX → 송신, 신호 레벨로 연결

결론

- **FIFO_RX push = rx_done, pop = ~tx_full / FIFO_TX push = ~rx_empty, pop = ~tx_busy**
- **uart_tx tx_start = ~FIFO_TX empty → FIFO에 byte가 있는 동안 계속 송신**



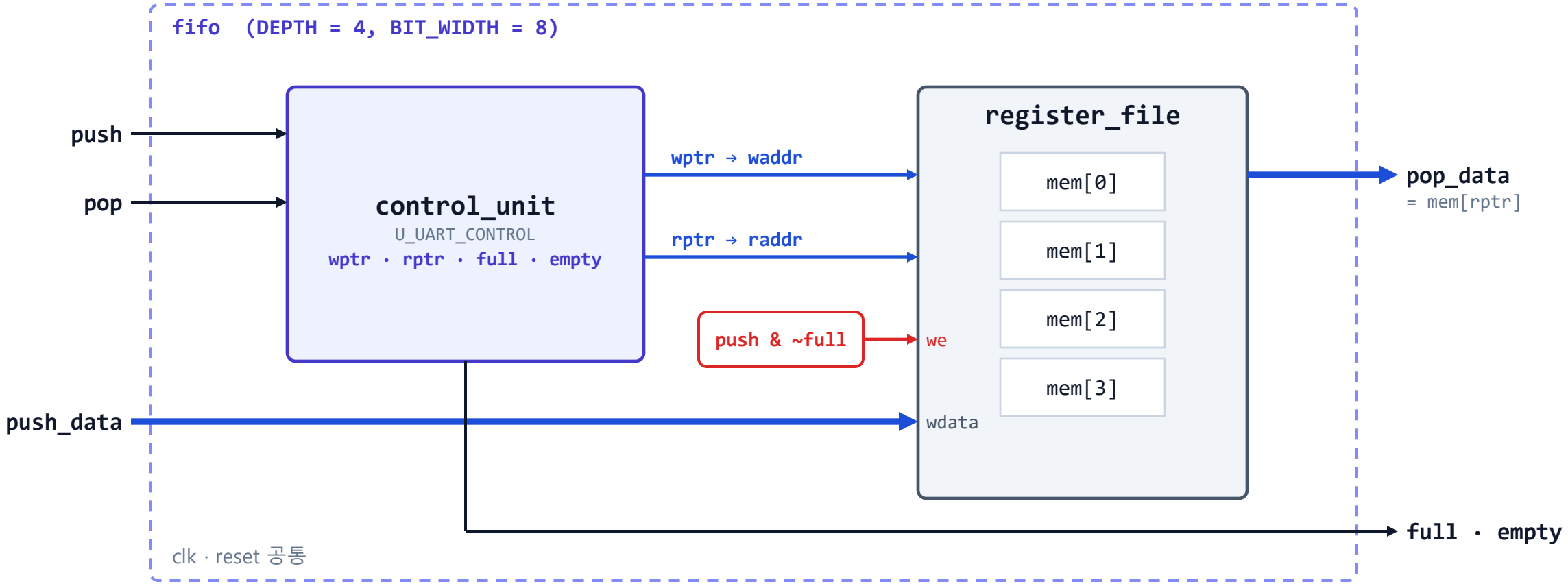
1. **push** rx_done 1 clk → FIFO_RX 저장
2. **전달** RX 비어있지 않고 TX 안 찼을 때 1 byte 이동
3. **tx_start** FIFO_TX에 byte가 있는 동안 1
4. **pop** TX가 바쁘지 않을 때 FIFO_TX에서 꺼냄

03

fifo = control_unit (포인터 · flag) + register_file (4 × 8 bit)

결론

- **control_unit: {push, pop}와 full · empty로 wptr · rptr · flag 갱신**
- **register_file: we = push & ~full일 때만 쓰기, 읽기 pop_data = mem[rptr] 조합 출력**



1 **control_unit** {push, pop} 조합별 포인터 · flag 갱신 (다음 장)

2 **register_file** 쓰기는 clk, 읽기는 조합 → FWFT

04

{push, pop} × empty · 중간 · full 동작

결론 · **full일 때 push 무시 · empty일 때 pop 무시**
· **push + pop 동시 — empty: push만 · 중간: 둘 다 · full: pop만 (구조상 spec)**

입력 / 상태	empty	중간 (1 ~ 3개)	full
push	wptr + 1 empty ← 0	wptr + 1 wptr = rptr일 경우 full ← 1	무시 (we = 0)
pop	무시	rptr + 1 rptr = wptr일 경우 empty ← 1	rptr + 1 full ← 0
push + pop	push만 wptr + 1 · empty ← 0	wptr + 1 · rptr + 1 개수 유지	**pop만 (spec)** **rptr + 1 · full ← 0**

1 **full + push·pop** we = push & ~full → 쓰기 차단 · pop만
그 clk 입력 byte는 버려짐 (spec)

2 **S1 검증** 9가지 경우 모두 기준 모델과 비교
→ mismatch 0 (414 step)

04

rx_done → tx 시작까지 clk 단위 흐름

결론

- **rx_done → FIFO_RX push → FIFO_RX pop · FIFO_TX push → tx_start → START → tx fall**
- **rx_done → tx fall 4 clk (40 ns) — 02 직결 구조 2 clk 대비 FIFO 2단 통과로 +2 clk**

E0
rx_done = 1
STOP 중앙 1 clk pulse

↓

E1
FIFO_RX push
wptr + 1 · empty ← 0

↓

E2
RX pop · TX push
같은 clk에 1 byte 이동

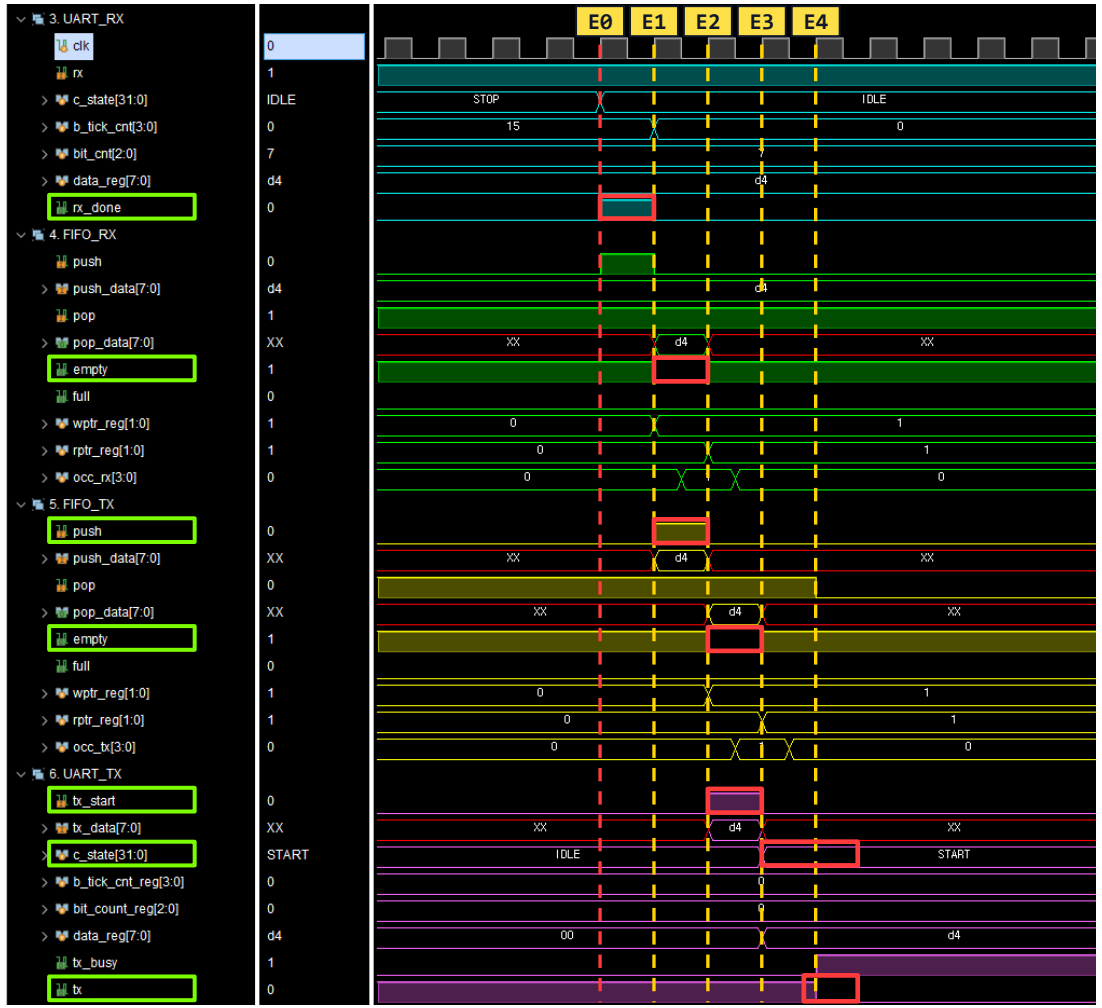
↓

E3
tx_start = 1
FIFO_TX empty = 0 → START

↓

E4
tx fall
START에서 tx ← 0

TB 로그 3 clk = rx_done을 negedge에서 기록한 값 (posedge 기준 4 clk)

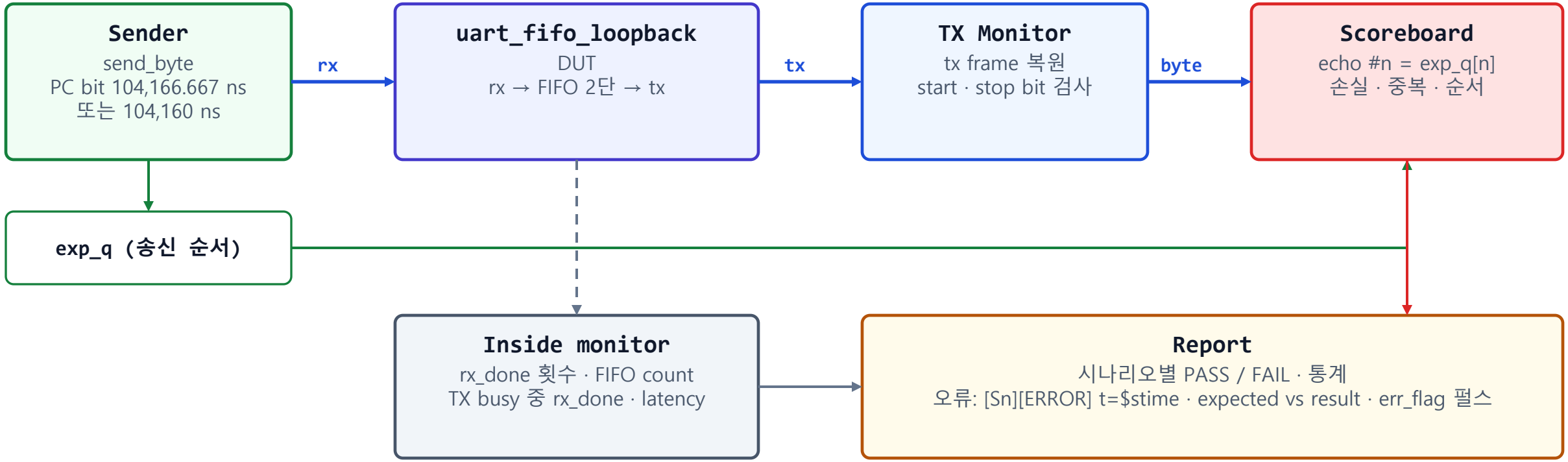


E0 → E4
4 clk (40 ns)

05

TB: Sender → DUT → TX Monitor → Scoreboard

결론 · 송신 byte를 queue에 저장 → tx에서 복원한 echo와 순서대로 비교 (손실 · 중복 · 순서 검출)
· FIFO는 TB 안 별도 instance (U_FIFO_UT) + 기준 모델로 매 clk 비교



S1 전용

U_FIFO_UT (fifo, DEPTH 4) ⇄ 기준 모델 (push는 full 아닐 때만 · pop은 empty 아닐 때만) → 매 step empty · full · pop_data 비교

UART FIFO 검증 시나리오 5개 모두 PASS

05

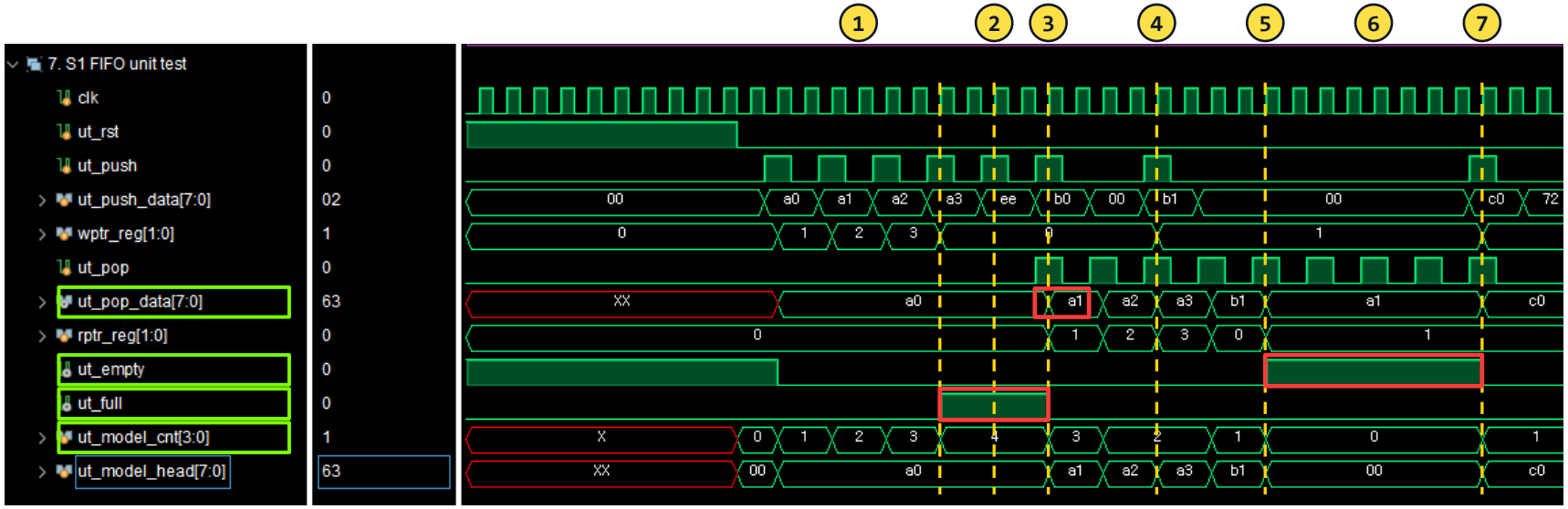
결론 · 시나리오 5개 — FIFO 단독 · 단일 byte · 9600 연속 · 최악 조건 · glitch 회귀
· PC 9600 bps 고정 전제, 최악 조건은 PC = FPGA 속도로 여유 0 tick 재현

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과	결과
S1	**FIFO 단독**	고정 시퀀스 + 무작위 400 step	ut_full · ut_empty · ut_pop_data vs 모델	불일치 0	**414 / 0**
S2	**단일 byte echo**	256 byte, byte마다 echo 대기 (bit 104,166.667 ns)	rx_done, FIFO count, tx	256 / 256	**256 / 256**
S3	**9600 bps 연속**	256 byte gap 없이 (bit 104,166.667 ns)	echo 순서 · 개수, FIFO count	손실 · 중복 0	**256 / 256**
S4	**최악 조건**	256 byte gap 없이 (bit 104,160 ns = FPGA 속도)	rx_done vs TX STOP→IDLE, FIFO_TX count	손실 · 중복 0	**256 / 256**
S5	**START glitch 회귀**	02 glitch 23 case	rx_done · echo 횟수	23 / 23	**23 / 23**

05

S1 FIFO : 414 Step 모두 기준 모델과 일치

결론 · 고정 시퀀스 (fill · full 중 push · full 중 push+pop · drain · empty 중 pop) + 무작위 400 step
· empty · full · pop_data mismatch 0 — full 중 push+pop은 pop만 (spec)



1 **fill** a0 ~ a3 push → wptr 0→1→2→3→0, full 1

2 **push & full** ee → 무시 (wptr · head 유지)

3 **push+pop & full** b0 → pop만 (full 0, spec)

4 **push+pop & mid** b1 → wptr · rptr +1, count 유지

5 **drain** pop 2번 → count 0, empty 1

6 **pop & empty** pop 3번 → 무시 (rptr 유지)

7 **push+pop & empty** c0 → push만 (empty 0)

이후 무작위 400 step · 매 step 모델과 일치

PASS

414 / 0

step / mismatch

확인한 경우 (횟수)

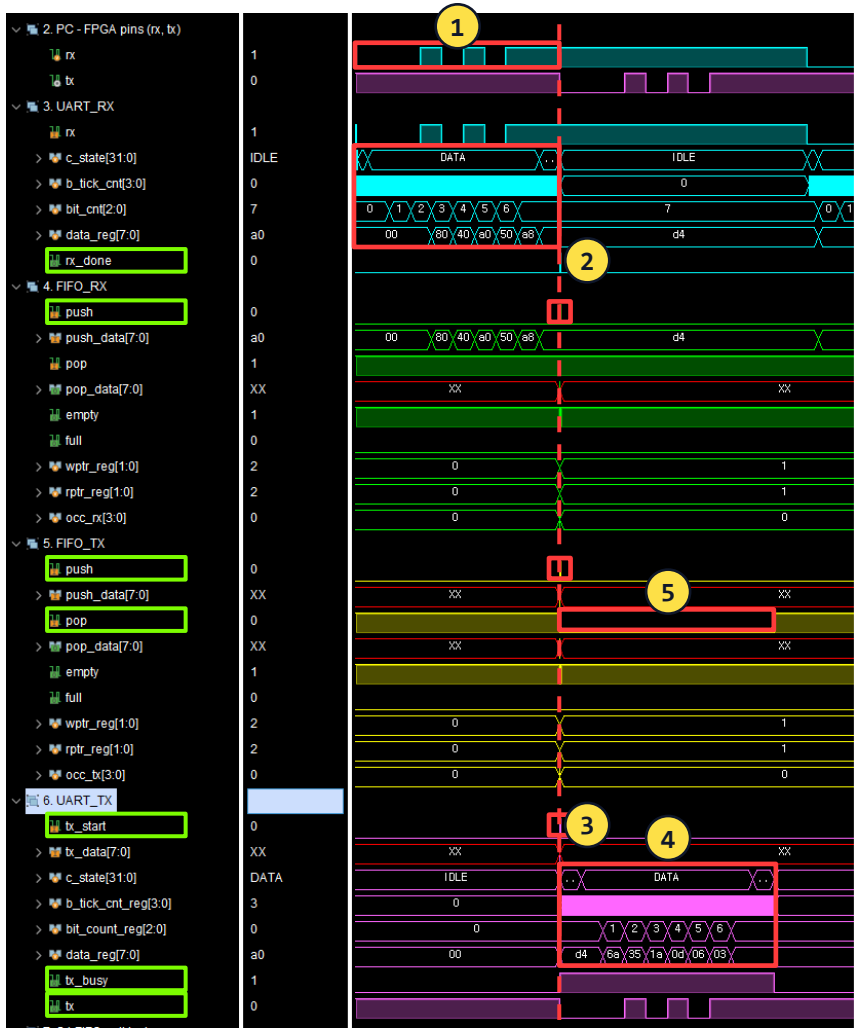
push & full	9
pop & empty	11
push+pop & empty	12
push+pop & mid	71
push+pop & full	8
wptr 순환	43

```
------------------------------------------------------------
[S1] FIFO unit : PASS
     steps 414 / mismatch 0
     covered : push@full 9, pop@empty 11, push+pop@empty 12,
               push+pop@mid 71, push+pop@full 8, wptr wrap 43
```

05

S2 단일 Byte Echo: FIFO 추가 후에도 256 / 256

결론 · 0x00 ~ 0xFF 무작위 순서 256 byte, byte마다 echo 확인 후 다음 송신 (PC bit 104,166.667 ns)
· echo 256 / 256 일치 · frame error 0



1. rx frame 수신 → data_reg 80 → … → d4
2. rx_done 1 clk → FIFO_RX push (빨간 선)
3. FIFO 2단 통과 → tx_start (4 clk)
4. uart_tx DATA: d4 → 6a → 35 … LSB부터 송신
5. tx_busy = 1 동안 FIFO_TX pop = 0

PASS

256 / 256

sent / echoed

TB 결과

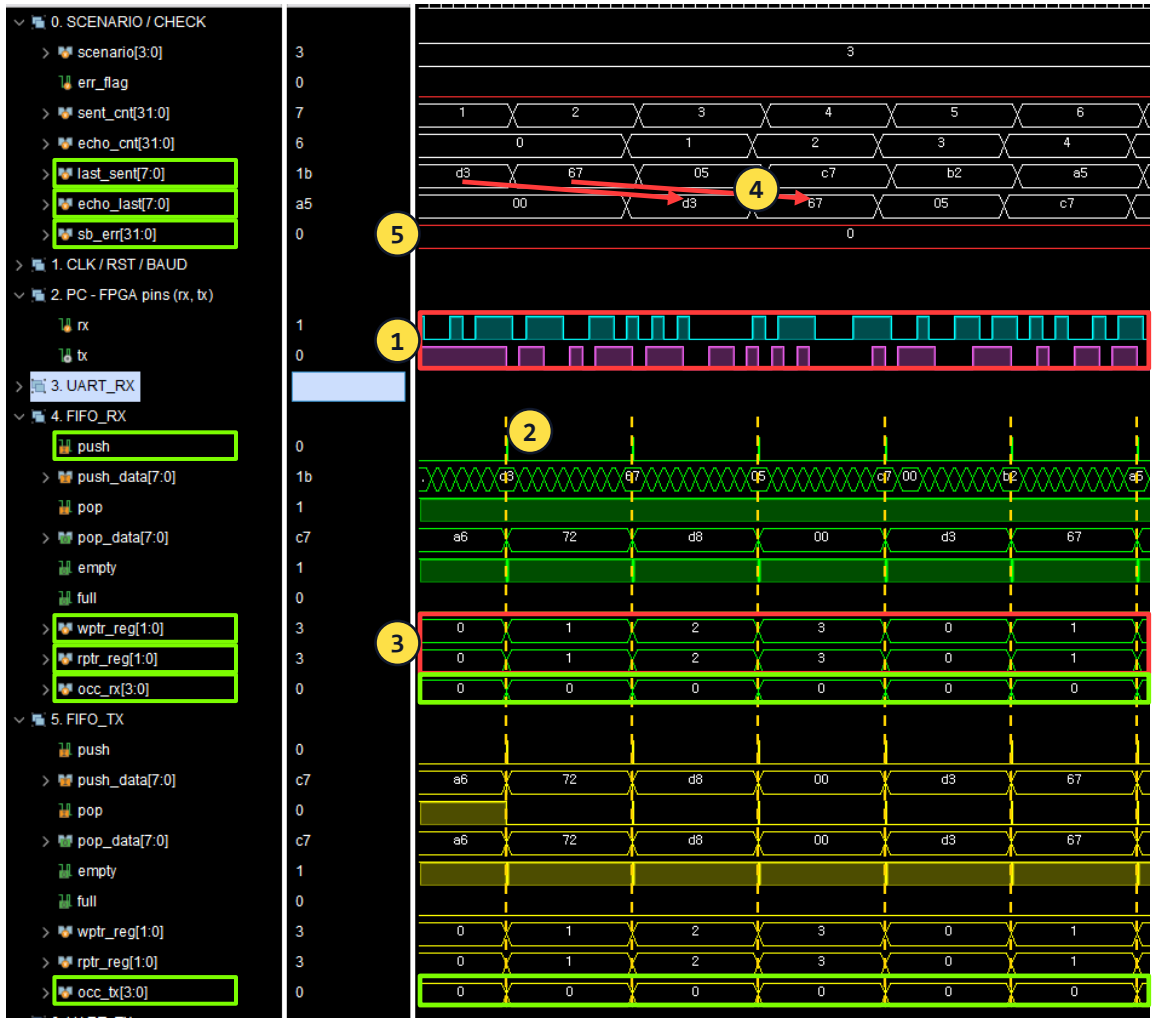
mismatch	0
frame error	0
rx_done → tx fall	4 clk
FIFO_RX / TX max	1 / 1

```
------------------------------------------------------------------
==== [S2] single byte echo : 256 bytes, wait for each echo (bit 104166.667 ns) ====
------------------------------------------------------------------
[S2] single byte echo (256) : PASS
     sent 256 / echoed 256 / mismatch 0 / frame error 0
     FIFO_RX max 1, FIFO_TX max 1, full RX 0 TX 0, rx_done while TX busy 0
     TX frame gap min 115227 clk, max 115878 clk
     rx_done -> tx fall latency min 3 clk, max 3 clk
------------------------------------------------------------------
```

S3 9600 bps 연속 256 Byte: 손실 · 중복 0

05

결론 · 256 byte를 gap 없이 연속 송신 (PC bit 104,166.667 ns) → echo 순서 · 개수 비교
· sent 256 = echoed 256 · mismatch 0



1. 연속 rx → 연속 echo
2. frame마다 FIFO push
3. wptr = rptr → count 0
4. echo 순서 = 송신 순서
5. sb_err 0 · err_flag 0

PASS

256 / 256

sent / echoed

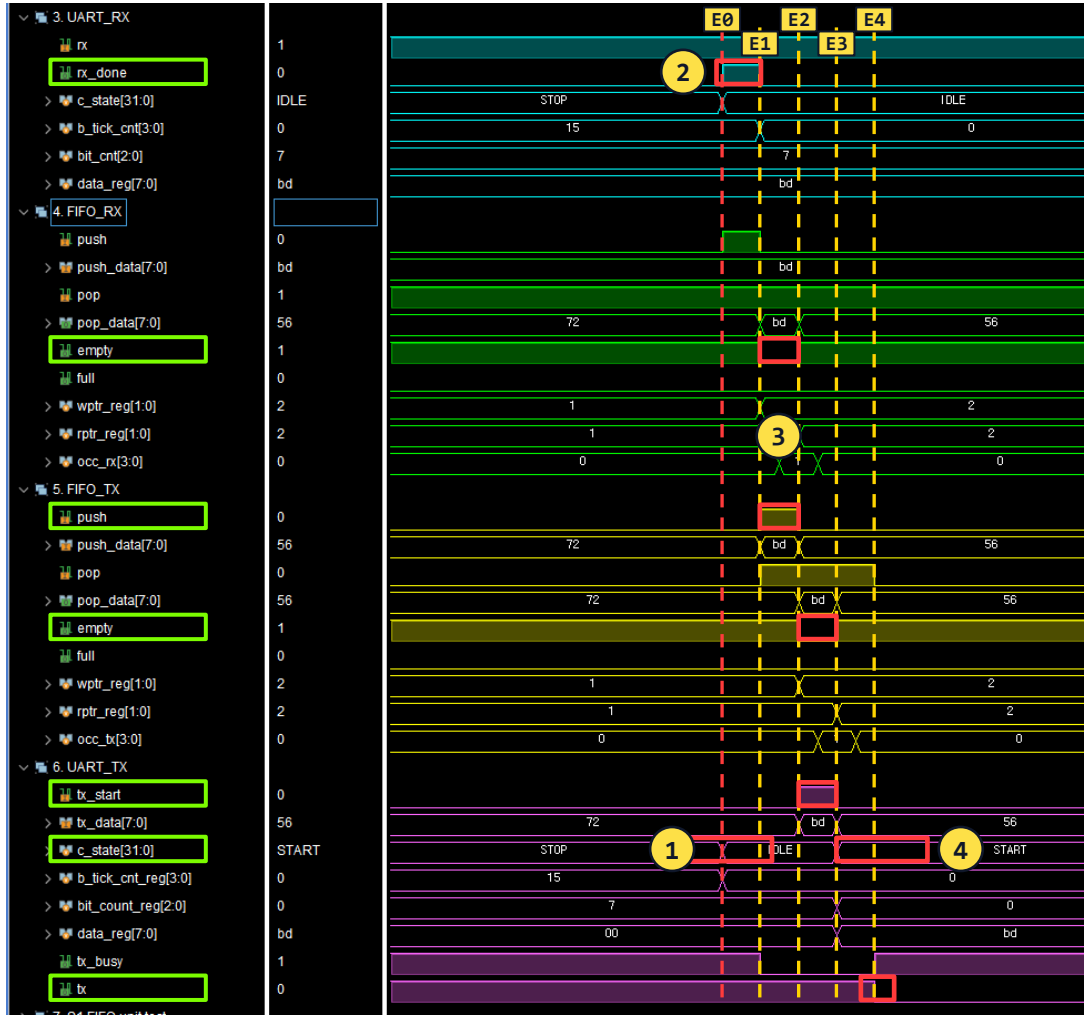
TB 결과

mismatch	0
FIFO_RX max	1
FIFO_TX max	1
full 횟수	0
TX busy 중 rx_done	0
TX frame gap	0 ~ 651 clk

S4 여유 0 tick 최악 조건: Frame 사이 공백 0 clk로 256 / 256

05

결론 · PC = FPGA 속도 (bit 104,160 ns) 연속 256 byte → rx_done 간격 = TX frame 길이, 여유 0 tick
· rx_done 순간 TX는 이미 IDLE (busy 중 rx_done 0회) → echo frame 공백 **0 clk** · **256 / 256 손실 0**



E0 rx_done ↑ = TX STOP → IDLE (같은 clk)

E1 FIFO_RX push · tx_busy 0

E2 FIFO_TX push · tx_start 1

E3 START · FIFO_TX pop

E4 tx fall → 다음 echo

PASS
256 / 256
sent / echoed

1 **STOP → IDLE**
앞 echo 종료

2 **rx_done**
TX는 이미 IDLE

3 **FIFO 2단 통과**
count 1 · 4 clk

4 **START**
공백 0 clk로 다음 echo

TX busy 중 rx_done	**0**
FIFO max / frame gap	**1 / 0 clk**

05

S5 START Glitch 23 Case: FIFO 구조에서도 유지

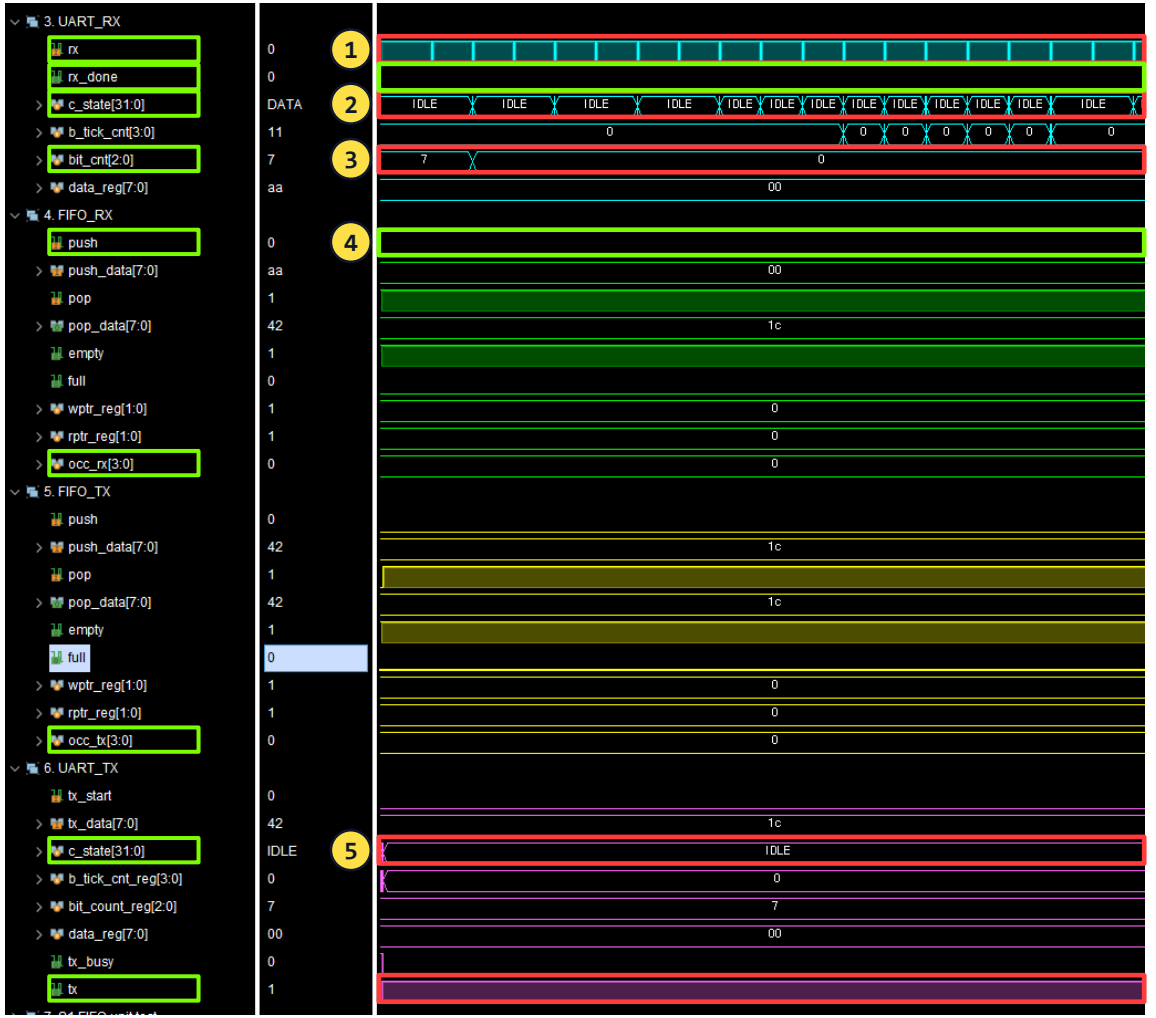
결론 · 02와 같은 glitch 23 case (single 16 · burst 2 · glitch 뒤 byte 5)
· **23 / 23 PASS** — uart_rx 동일, 02 트러블 슈팅 결과 유지

구조	START 처리	결과
02 loopback (수정 전)	rx 재확인 없음	6 / 23
02 loopback (수정 후)	START 중 rx = 1 → IDLE	23 / 23
04 FIFO loopback	같은 uart_rx	23 / 23

1. rx glitch 입력 (가는 low pulse 반복)
2. c_state IDLE → START → IDLE 복귀
3. bit_cnt 0 유지 → DATA 진입 없음
4. rx_done 0 · FIFO push 0 · count 0
5. UART_TX IDLE · tx = 1 → echo 없음

```
------------------------------------------------------------------------
[S5] START glitch regression : PASS
     pass 23 / fail 0
------------------------------------------------------------------------

=================== SUMMARY ===================
 S1 FIFO unit                : PASS  (414 steps)
 S2 single byte echo         : PASS  (256 / 256)
 S3 9600 bps back-to-back    : PASS  (256 / 256)
 S4 worst case (PC = FPGA)   : PASS  (256 / 256)
 S5 START glitch regression  : PASS  (23 / 23)
================================================
```



검증 중 확인한 경계 동작 2가지

06

결론 · 시나리오 5개 모두 PASS → FAIL로 이어진 문제 없음
· 구조상 동작 2가지 확인 — full 중 push+pop은 pop만 · FIFO_TX pop = ~tx_busy

1 full + push · pop 동시 → pop만 (구조상 spec)

- register_file we = push & ~full → full 상태에서는 쓰기 차단
- control_unit은 pop만 진행 → 그 clk의 입력 byte는 버려짐
- 시스템에서는 FIFO_RX가 full일 때만 해당
- S1에서 push+pop@full 경우를 모델과 비교 → 일치

S2 ~ S4 full 횟수	**0**
S1 push+pop@full 횟수	**8**

2 FIFO_TX pop = ~tx_busy

- tx_busy는 START 진입 다음 clk에 1
- TX가 IDLE에서 시작하는 순간 FIFO_TX에 2개가 있을 경우 추가 pop 가능
- PC 9600 고정 → FIFO_TX 최대 1개 (S2 ~ S4 실측) → 재현 조건 없음
- S2 ~ S4 echo 손실 0으로 영향 없음 확인

S2 ~ S4 FIFO_TX max	**1**
echo 손실	**0**

07

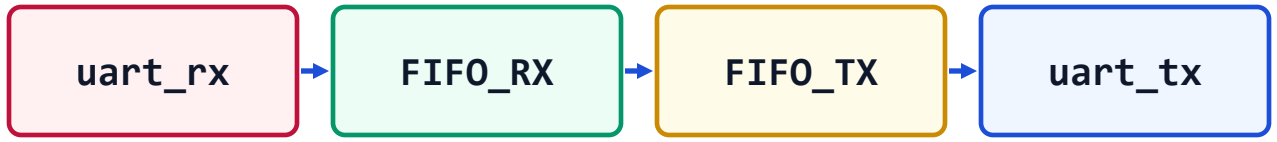
FIFO 2단 구조로 연속 입력 Echo 손실 0

결론 · 시나리오 5개 모두 PASS — FIFO 단독 · 단일 byte · 9600 연속 · 최악 조건 · glitch 회귀
· PC 9600 고정에서 FIFO는 대기 없이 통과 (최대 1개) · 지연 +2 clk → 연속 입력 echo 손실 0

결론

시나리오	결과	판정
S1 FIFO 단독	414 step · mismatch 0	PASS
S2 단일 byte echo	256 / 256	PASS
S3 9600 bps 연속	256 / 256	PASS
S4 최악 조건	256 / 256	PASS
S5 START glitch	23 / 23	PASS

구조 요약



고찰

- PC 9600 고정 → TX busy 중 rx_done 0회, FIFO 최대 1개 → 보관 기능은 이 조건에서 쓰이지 않음 (PC가 더 빠를 경우 대비)
- full · overflow는 시스템 입력으로 재현 불가 → FIFO 단독 TB (S1)로 검증
- FIFO_RX full 미연결 → overflow 발생 시 표시 없음

개선 방향

1. FIFO_TX pop을 uart_tx가 tx_data를 받는 순간과 일치시키는 handshake
2. full · overflow flag를 top 출력으로 → 손실 감지
3. DATA 구간 3 sample 다수결 (02 고찰 유지)

Q & A

감사합니다

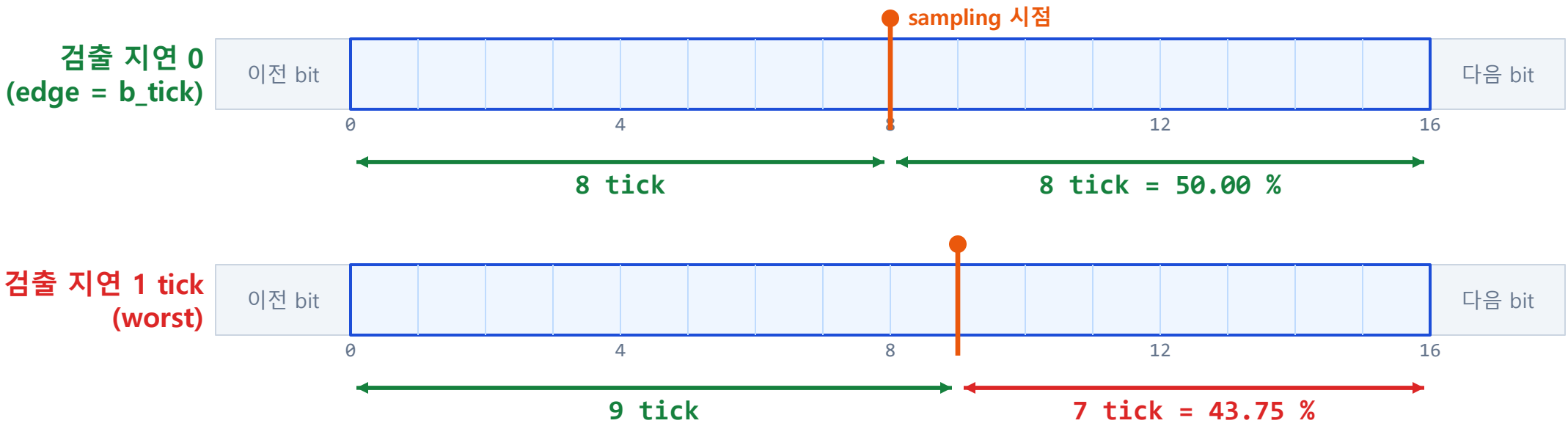
정안성

A

Sampling Margin: Start 검출 지연 최대 1 tick → 늦은 쪽 43.75 %

결론

- **Margin = sampling 시점에서 bit 경계까지 남은 거리, bit 중앙 sampling = 양쪽 8 tick (50 %)**
- **rx fall은 b_tick 순간에만 검출 → 0 ~ 1 tick 늦게 START → 늦은 쪽 7 tick = 43.75 %**



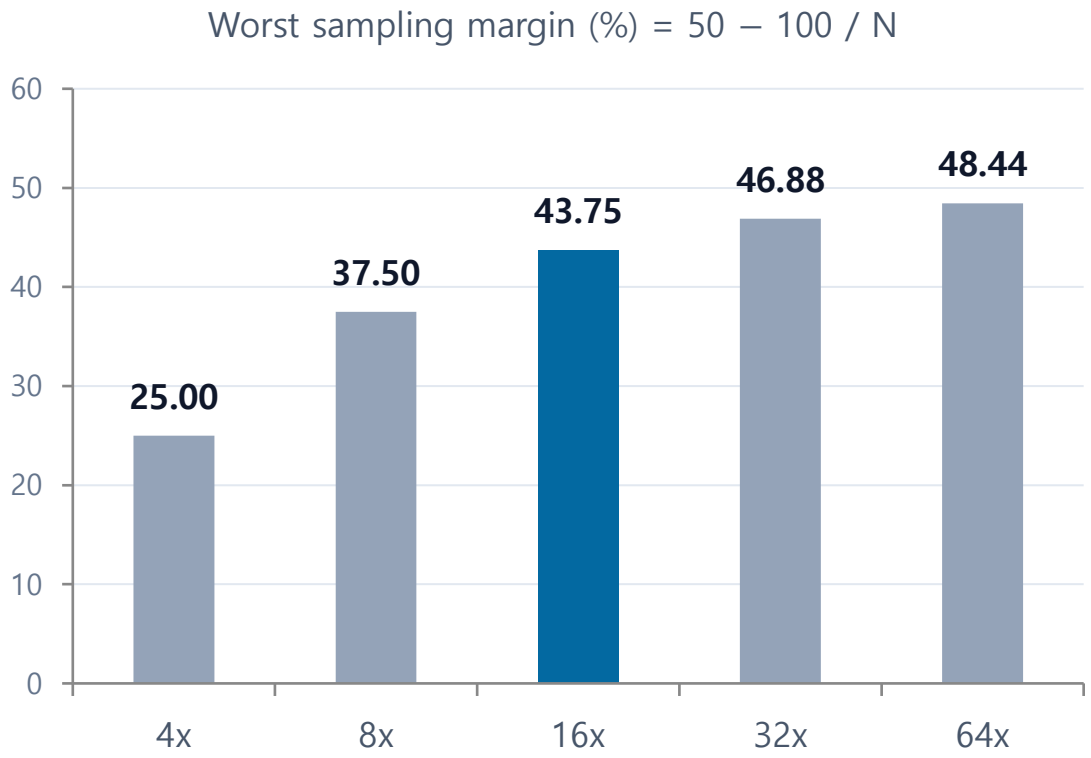
1. **지연 원인** b_tick = 1 순간만 rx 확인 → 최대 1 tick 늦게 START
2. **늦은 쪽** sample이 오른쪽으로 밀림 → 다음 bit까지 7 tick (43.75 %)
3. **이른 쪽** 왼쪽 여유는 8 ~ 9 tick → 최소 50 % 유지
4. **누적 X** START마다 falling edge로 다시 정렬 → 지연은 frame마다 초기화, xsim에서 모든 frame rx fall → rx_done 989.525 µs 동일

A

16x: Margin · Baud 정확도 · 최대 속도의 균형점

결론

- **Oversampling N을 키워도 worst margin = 50 − 100/N %로 조금씩만 증가 (16x 43.75 % → 32x 46.88 %)**
- **N이 클수록 F_COUNT 정수 내림 오차 ↑ (32x +0.16 %) · 같은 clk에서 최대 baud = clk ÷ N ↓**



F_COUNT = 100 MHz ÷ (9600 × N), Verilog 정수 나눗셈 내림

N	F_COUNT	실제 baud	오차	최대 baud
4x	2604.17 → 2604	9600.61	+0.0064 %	25.00 Mbps
8x	1302.08 → 1302	9600.61	+0.0064 %	12.50 Mbps
16x	**651.04 → 651**	**9600.61**	**+0.0064 %**	**6.25 Mbps**
32x	325.52 → 325	9615.38	+0.16 %	3.125 Mbps
64x	162.76 → 162	9645.06	+0.47 %	1.563 Mbps

1 **16x** margin 43.75 % · 오차 +0.0064 % · 최대 6.25 Mbps
→ 9600 bps에 충분, 업계 표준 배수

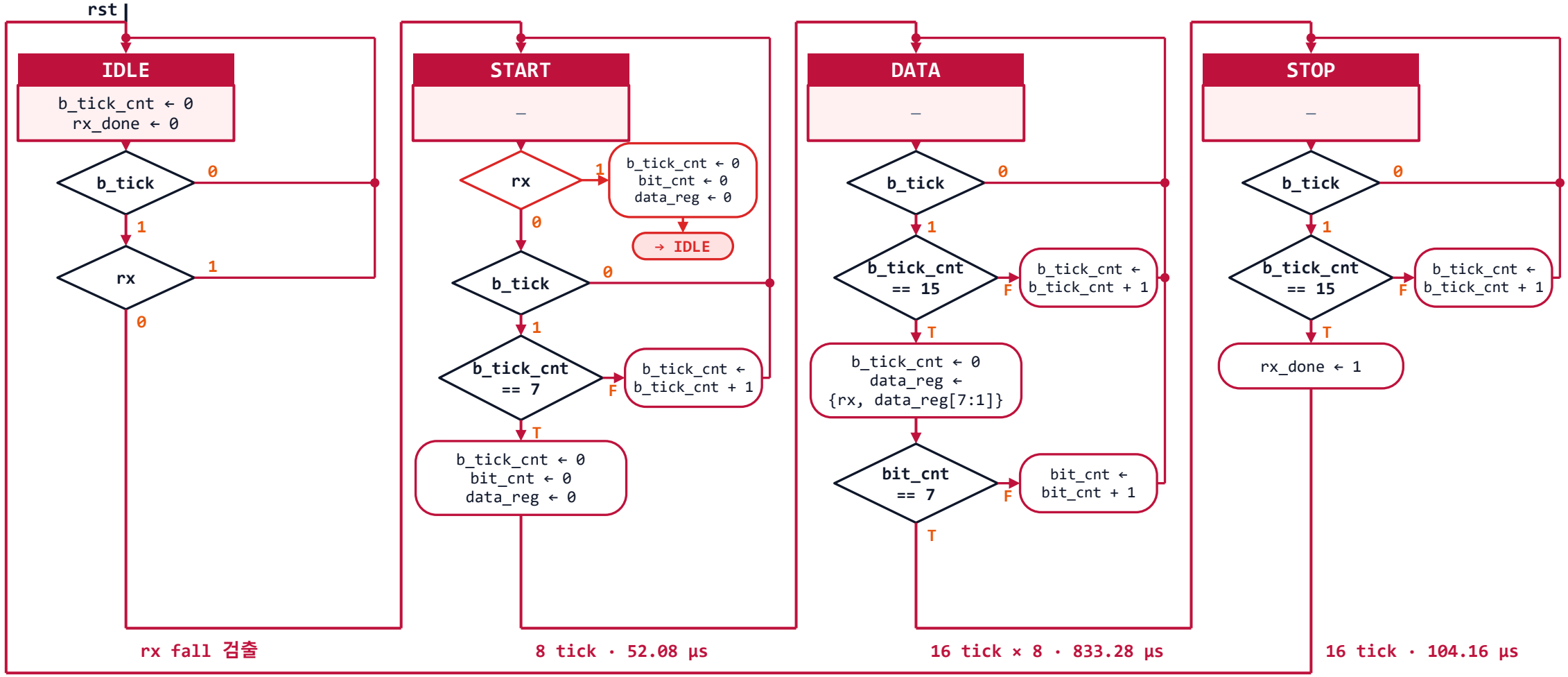
2 **더 키울 경우** 32x: margin +3.1 %p 대신 baud 오차 25배 · 최대 속도 절반 → 노이즈 대책은 N보다 sample 다수결

A

uart_rx ASM: START 중앙 정렬 후 16 tick마다 Sampling

결론

- rx fall 검출 → START 8 tick 동안 rx = 0 유지 확인, rx = 1일 경우 IDLE 복귀 (glitch 거름) → bit 중앙 도달
- DATA에서 16 tick마다 1 bit shift (8회) → STOP 중앙에서 rx_done 1 clk



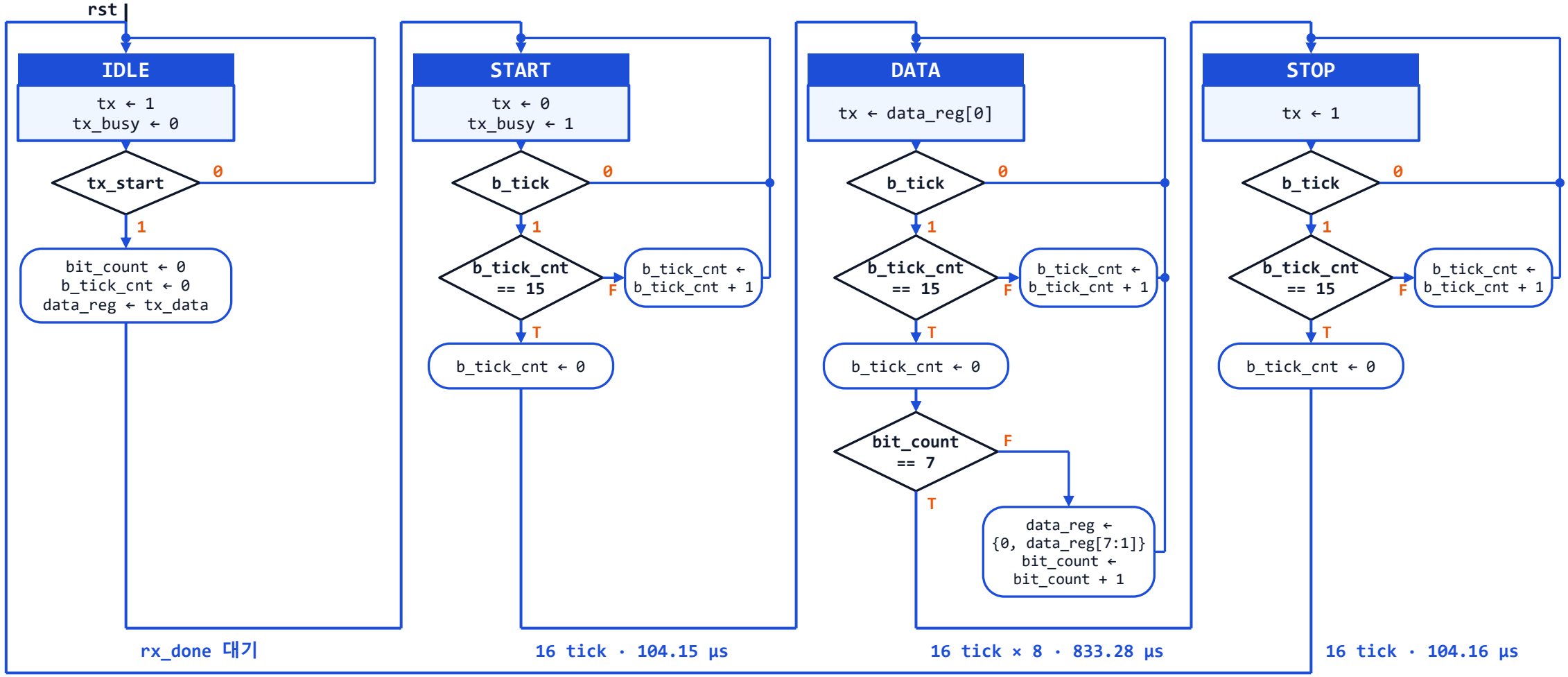
State box Decision box Conditional output ● path merge ← 다음 clock edge 갱신 · T / F 조건 참 / 거짓

A

uart_tx ASM: tx_start 후 각 Bit 16 tick 유지, LSB First

결론

- IDLE에서 tx_start = 1 → tx_data latch → START(0) · D0 ~ D7 · STOP(1) 각 16 tick 송신
- tx_start는 IDLE에서만 확인, tx_busy는 START ~ STOP 동안 1



State box · Decision box · Conditional output · ● path merge · ← 다음 clock edge 갱신 · T / F 조건 참 / 거짓

A

TB 로그: 시나리오 5개 모두 PASS

결론

- **S1 414 step mismatch 0 · S2 ~ S4 256 / 256 · S5 23 / 23**
- **S2 ~ S4 FIFO 최대 1 · full 0 · TX busy 중 rx_done 0 · latency 로그 3 clk (negedge 기준, posedge 기준 4 clk)**

```
------------------------------------------------------------
[S1] FIFO unit : PASS
     steps 414 / mismatch 0
     covered : push@full 9, pop@empty 11, push+pop@empty 12,
               push+pop@mid 71, push+pop@full 8, wptr wrap 43
------------------------------------------------------------------------
==== [S2] single byte echo : 256 bytes, wait for each echo (bit 104166.667 ns) ====
------------------------------------------------------------------------
[S2] single byte echo (256) : PASS
     sent 256 / echoed 256 / mismatch 0 / frame error 0
     FIFO_RX max 1, FIFO_TX max 1, full RX 0 TX 0, rx_done while TX busy 0
     TX frame gap min 115227 clk, max 115878 clk
     rx_done -> tx fall latency min 3 clk, max 3 clk
------------------------------------------------------------------------
------------------------------------------------------------------------
[S5] START glitch regression : PASS
     pass 23 / fail 0
------------------------------------------------------------------------

=================== SUMMARY ====================
 S1 FIFO unit                 : PASS  (414 steps)
 S2 single byte echo          : PASS  (256 / 256)
 S3 9600 bps back-to-back     : PASS  (256 / 256)
 S4 worst case (PC = FPGA)    : PASS  (256 / 256)
 S5 START glitch regression   : PASS  (23 / 23)
================================================
```

```
------------------------------------------------------------------------
==== [S3] 9600 bps back-to-back : 256 bytes, no gap (bit 104166.667 ns) ====
------------------------------------------------------------------------
[S3] 9600 bps back-to-back (256) : PASS
     sent 256 / echoed 256 / mismatch 0 / frame error 0
     FIFO_RX max 1, FIFO_TX max 1, full RX 0 TX 0, rx_done while TX busy 0
     TX frame gap min 0 clk, max 651 clk
     rx_done -> tx fall latency min 3 clk, max 3 clk
------------------------------------------------------------------------
==== [S4] worst case : 256 bytes, no gap, PC = FPGA speed (bit 104160 ns) ====
------------------------------------------------------------------------
[S4] worst case back-to-back (256) : PASS
     sent 256 / echoed 256 / mismatch 0 / frame error 0
     FIFO_RX max 1, FIFO_TX max 1, full RX 0 TX 0, rx_done while TX busy 0
     rx_done -> tx fall latency min 3 clk, max 3 clk
```